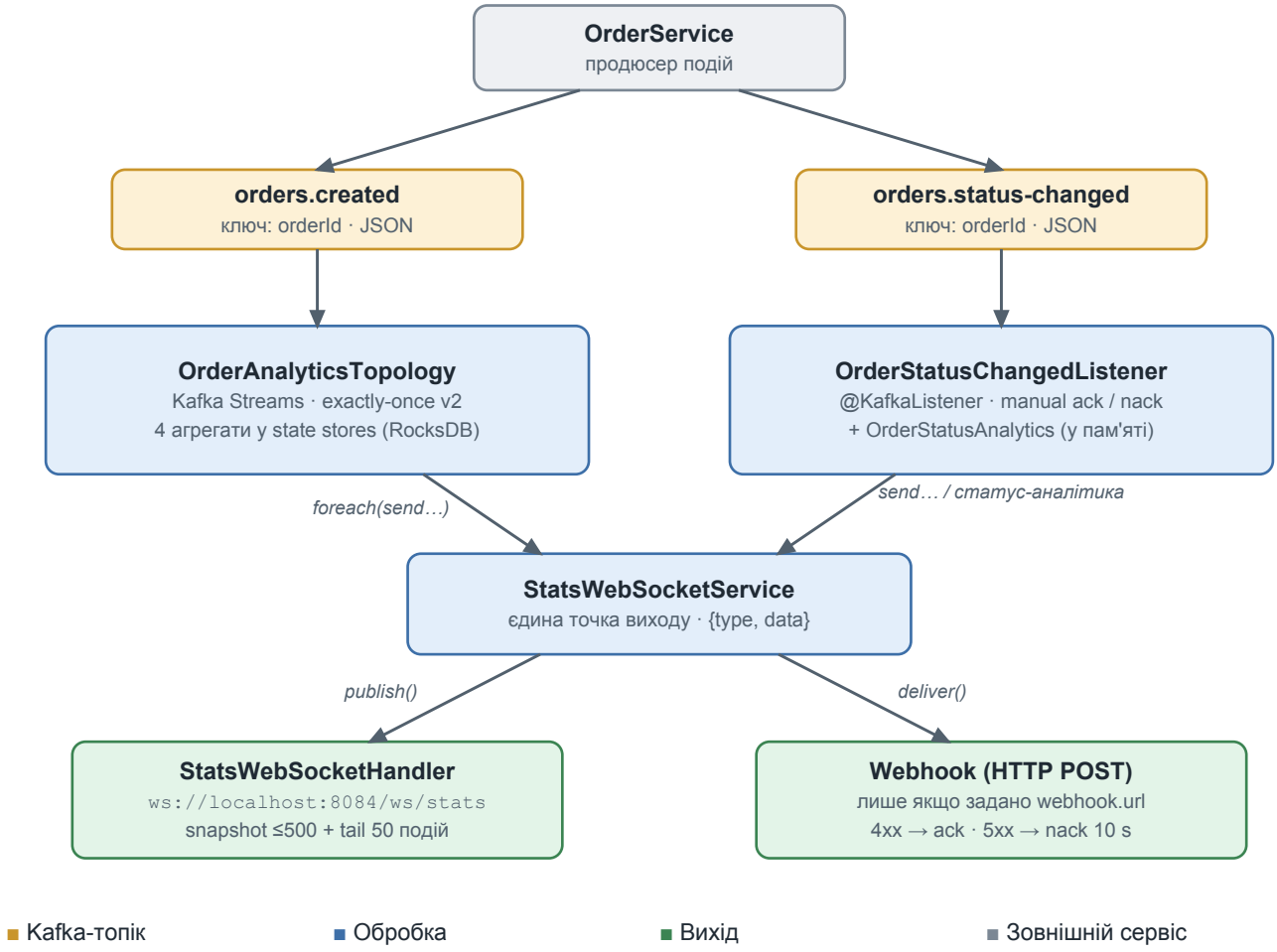
AnalyticsService — Data Flow

Технічний звіт · 10.11.2026 · модуль **AnalyticsService** (Spring Boot 4.1, Kafka Streams, WebSocket)

> **Коротко.** Сервіс має два входи з Kafka (`orders.created` і `orders.status-changed`) та одну точку виходу — `StatsWebSocketService`. Звідти дані йдуть у браузерний дашборд через WebSocket і, опційно, на зовнішній webhook. Назад у Kafka результати не пишуться.

1. Загальна схема



2. Потік 1: orders.created → Kafka Streams

Файл: `config/OrderAnalyticsTopology.java:172`

1. **Вхід.** Топологія читає `orders.created`. Ключ — `orderId`, значення — JSON, що десеріалізується в `OrderCreatedEventDto` (`orderId, customerId, productId[], currency`). Serde називається `orderAvroSerde()`, але фактично це Jackson JSON (`KafkaStreamsConfig.java:55`).
2. **Чотири незалежні гілки.** З одного стріму будуються чотири агрегати (таблиця нижче).
3. **Вихід.** Кожна гілка завершується `.toStream().foreach(webSocketService::sendX)` — оновлене значення агрегату одразу передається у сервіс виходу.

Гілка	Групування	State store	Тип повідомлення → графік
Статистика клієнта	`customerId` (фільтр: є товари)	`customer-order-stats-store` Avro `CustomerOrderStats`	`CUSTOMER_STATS` — замовлення, товари, середнє на замовлення

Гілка	Групування	State store	Тип повідомлення → графік
Темп замовлень	константа `"ALL"` + tumbling-вікно `PT1M`	`orders-per-window-store`	`ORDER_RATE` → line chart (ключ — початок вікна)
Валюти	`currency`	`currency-stats-store`	`CURRENCY_BREAKDOWN` → donut
Популярність товарів	`flatMap` по `productId`	`product-stats-store`	`PRODUCT_POPULARITY` → bar chart

Важливо про гарантії. `EXACTLY_ONCE_V2` (`KafkaStreamsConfig.java:50`) захищає лише state stores. Відправка у WebSocket та webhook — побічний ефект: при rebalance чи рестарті дашборд може отримати повторне значення. Це безпечно, бо кожне повідомлення несе повний агрегат, а не приріст.

3. Потік 2: orders.status-changed → Kafka consumer

Файл: `listener/OrderStatusChangedListener.java:326`

1. **Вхід.** `@KafkaListener` отримує `OrderStatusChangedEventDto`: попередній і новий статус, причина, хто змінив, `occurredAt`. Статуси — рядки, тому новий статус в OrderService не зламає десеріалізацію.
2. **Валідація.** Без `orderId` або `newStatus` подія логується і відкидається з ack.
3. **Аналітика** (`analytics/OrderStatusAnalytics.java:766`). Помилка тут не блокує пересилання події.
4. **Пересилання події** `ORDER_STATUS_CHANGED` у дашборд і на webhook.

Що рахує OrderStatusAnalytics

Метрика	Як рахується	Тип повідомлення
Час між статусами	Лише якщо цей інстанс бачив вхід у попередній статус; count / avg / max / last у секундах	`STATUS_TRANSITION_TIME`
Воронка	Частка унікальних замовлень, що побували в кожному статусі	`STATUS_FUNNEL`
Причини	Топ-5 причин переходу для кожного статусу	`STATUS_REASONS`
Дедуплікація	Ключ `newStatus|occurredAt` на замовлення; LRU до `10 000` замовлень	—

Обробка результату webhook

Результат	Дія	Чому
Успіх	`ack`	Подію доставлено.
4xx (`HttpClientErrorException`)	`ack`, подію відкинуто	Повтор тих самих байтів не допоможе — лише заблокує партицію.
5xx / timeout / інше	`nack(10s)`	Тимчасова помилка — повторна доставка.

Чому не Kafka Streams? Consumer може утримувати offset, поки webhook недоступний. У Streams виняток зупиняє потік і змушує відмотати весь стан обробки. Доставка — at-least-once без дедуплікації, тож підписник webhook має орієнтуватися на `orderId` + `occurredAt`.

4. Вихід: StatsWebSocketService

Файл: `service/StatsWebSocketService.java:503`

- **Дашборд:** `StatsWebSocketHandler.publish()` серіалізує `{type, data}` і розсилає всім відкритим сесіям.
- **Webhook:** POST `{type, data}` на `webhook.url`. Порожній URL — webhook вимкнено, залишається лише WebSocket.
- **Помилки:** для агрегатів — лише лог (наступний агрегат все одно перекриє значення). Для `ORDER_STATUS_CHANGED` — виняток іде в listener, щоб спрацював `nack`.

5. Що отримує дашборд при підключенні

Файл: `websocket/StatsWebSocketHandler.java:652`

- **Snapshot** — останнє значення кожного агрегату за ключем `type|key`, не більше 500 записів (витісняється найдовше не оновлюваний).
- **Event tail** — останні 50 подій `ORDER_STATUS_CHANGED`.
- Новий клієнт одразу отримує snapshot + tail, тож дашборд не порожній до наступного замовлення.
- Доступ обмежено лише allowlist origin (`analytics.websocket.allowed-origins`). Автентифікації немає — ендпоінт не слід відкривати поза мережею compose.

6. Де живе стан

Стан	Сховище	Після рестарту
Агрегати Kafka Streams	RocksDB + changelog-топіки	Відновлюється
OrderStatusAnalytics	Пам'ять JVM	Обнуляється
Snapshot / event tail дашборду	Пам'ять JVM	Порожній до нових подій

При кількох інстансах кожен бачить лише свої партиції. Оскільки ключ — `orderId`, усі переходи одного замовлення потрапляють на один інстанс, але воронка і метрики часу будуть частковими на кожному з них.

7. Виявлені проблеми

Ці пункти, найімовірніше, заважають запуску сервісу в поточному стані.

#	Проблема	Наслідок	Де
1	У `application.properties` лише `spring.application.name`. Немає `kafka.topics.orders-status-changed`, `analytics.websocket.allowed-origins`, manual ack mode.	Падіння при старті на `@Value`; без manual ack `Acknowledgment` не інжектиться.	`src/main/resources/`
2	Namespace в Avro-схемі — `com.project.analyticsservice.avro`, а код імпортує `analyticsservice.avro.CustomerOrderStats`.	Помилка компіляції.	`CustomerOrderStats.avsc`
3	Дублікати пакетів: `config/config/`, `config/dto/config/…` тощо.	Дубльовані Spring-біни / конфлікти класів; ймовірно, випадкове копіювання.	`src/main/java/…/config/`
4	Топіки `orders.stats` і `orders.windowed-count` створюються, але в них ніхто не пише.	Мертва конфігурація.	`KafkaTopicConfig.java`